

# The Art of Concert Programming

Jared Gentner  
Boston, MA, USA  
jagen315@gmail.com

## Abstract

The process of preparing a program for a live performance is an extremely fulfilling one, not only because it can be artistically and spiritually rewarding, but also because it can be a technical challenge much like designing computer programs or systems. Performances may embed various art forms, such as song, dance, lighting and stage-play. The mediums for coordinating such performances are varied and rarely precise. We construct and describe Programmart, a specification language for writing concert programs, which leverages programming language features to handle the intricacies of crafting a live performance.

**CCS Concepts:** • General and reference → General conference proceedings; • Applied computing → Sound and music computing.

**Keywords:** Music

## ACM Reference Format:

Jared Gentner. 2026. The Art of Concert Programming. In *Proceedings of FARM '26*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3759162.3759648>

## 1 Introduction

Concert programs are used across all sorts of events, ranging from recitals to musicals to religious services. Traditional workflows for constructing concert programs involve importing rendered media into word processing or publishing software. If the program contains music, rendered score snippets come either from score editors or from scans or screenshots of published scores. Generating multiple variants of one program usually involves maintaining full copies of the program. If the music varies, copies of the scores also have to be maintained.

The Programmart language is designed to alleviate the aforementioned pain points of traditional workflows. More

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org). *FARM '26, Indianapolis*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-2145-8/25/10

<https://doi.org/10.1145/3759162.3759648>

ambitiously, it enables a new way of thinking about the relationship between performance and notation of performance. It does this by facilitating the creation of modular, extensible performance specifications. We will demonstrate this by completing the following tasks in Programmart:

1. Zooming in on and playing back parts of a concert program.
2. Producing multiple different renderings of the same concert program in a maintainable way.
3. Composing concert programs from sub-programs.

## 2 Exemplar: PorchFest 2026 Concert Program

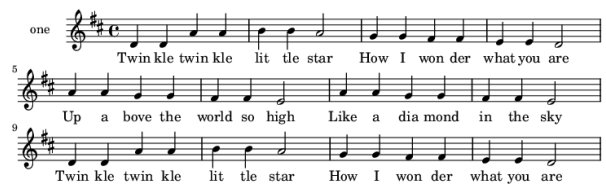
Rather than starting with a computer program, we will work backwards from a rendered concert program whose source is in Programmart. This program was performed as an opening ceremony for Somerville PorchFest 2026<sup>1</sup>. The score calls for violin, viola, cello, and theremin. It begins with a section called "Look mom, I can play a string instrument"<sup>2</sup>, consisting of a trio of nursery rhymes. Next is the surreal "Dream Sequence" section featuring some stageplay and a dance around a theremin. The performance concludes with the "Celebration" section, featuring a Traditional Irish Jig.

The concert program is rendered in full, as follows:

```
(realize (scribble-realizer)
  porchfest-program)
```

### Look mom, I can play a string instrument Twinkle Twinkle Little Star.

*Maria gives an introduction.*



### Hot Cross Buns.

*Jared gives an introduction.*



<sup>1</sup><https://www.somervilleporchfest.org/>

<sup>2</sup>The section is called this because two of the performers were playing string instruments despite not being trained on them.

## Row Row Row Your Boat.

Cat gives an introduction.



## Dream Sequence

**MARIA.** I'm tired...

*Maria goes over to the side of the porch and lays down. A blanket is tossed over her.*

*Tim, cloaked in the garbs of death, enters from stage rear, with a theremin and a folding chair.*

*Tim tunes the theremin while Cat and Jared lift Maria up. Maria is in a daze.*

*A random audience member is selected onstage from the audience. That audience member is Mariette.*

*Mariette, Maria, Jared, and Cat dance around the Theremin as the music gets more and more intense.*

*Maria falls to the ground. A pause.*

**MARIETTE.** I think she's dead.

**JARED.** And this is what happens when you [REDACTED].  
The good people of [REDACTED] porch would like to remind you: No [REDACTED].

[REDACTED] Tim [REDACTED]  
[REDACTED], then exits stage rear.

*Prep for swallowtail. Jared and Mariette do World's Smallest Violin on loop while this happens.*



## Celebration

### Swallowtail Jig.



The PorchFest program's Programmart source is compiled into Scribble [6], the Racket documentation language, by Programmart's Scribble realizer, which is invoked by the `realize` form. During this process, the score snippets are compiled to Lilypond [1], a music engraving domain specific language. The Lilypond is rendered into images to be loaded in the output Scribble. Finally, the output Scribble is embedded into this document, which is also written in Scribble. Scribble compiles into PDF, HTML, or LaTeX, which are all viable targets for concert programs.

## 3 The Programmart Language

### 3.1 Programmart Syntax

We introduce the syntax for Programmart. We will use a fixed syntax; however, it is important to note that Programmart is an extensible language. In fact, Programmart is one of a family of extensible languages called Art languages. Art languages share four core classes of forms: *coordinates*, *objects*, *rewriters*, and *embeddings*.

*Coordinates* denote locations within a program. Programmart programs have two coordinates:

```
<coord> ::= ( section <id>* )
          | ( index <number>* )
```

The `section` coordinate divides the program into hierarchical sections. Within those sections, elements are ordered using the `index` coordinate.

*Objects* represent the various elements of the program. Some objects are rendered. Some objects provide context for other objects. Our Programmart is rather sparse in objects:

```
<object> ::= ( layout <id>* )
             | ( art-title <id>* )
```

The `layout` object contains section names and specifies the order of the program's sections. The `art-title` object declares a title for a program section.

*Rewriters* are forms that do not get written into the concert program at all. Rather their purpose is to, when evaluated, transform the program in some way. Our Programmart has these rewriters:

```
<rewriter> ::= ( append-section <id>
                  #:after <id> )
               | ( insert-section <id>
                  #:before <id> )
```

This pair of rewriters is used to compose one concert program into another. They will be exercised in a later section.

*Embeddings* are forms that act as boundaries between a host Art language and an embedded Art language. Programmart features embeddings prominently. It achieves multimedia composition by embedding Art language representations of the media, rather than a rendered output *à la* traditional workflows. Our demo Programmart hosts two embedded Art languages:

```
| ( rewrite-in-script <rewriter>* )
```

```
<embedding> ::= ( music <tonart-form>* )
              | ( script <spielart-form>* )
```

The **music** form contains a Tonart [3] program. Tonart is a language for composing music scores. See "Look mom, I can play a string instrument" for an example of rendered Tonart. The **script** form contains a Spielart program. Spielart is a language for writing play scripts. See "Dream Sequence" for rendered Spielart.

All Art languages share this form:

```
<art-form> ::= <coord>
              | <object>
              | <rewriter>
              | <embedding>
              | ( @ ( <coord>* ) <art-form>* )
```

The @-form does two things. An @-form surrounding an object attaches its coordinates to the object. Objects' attached coordinates are used to figure out whether two objects relate to each other or not. For example, an object in (section foo) would not be related to an object in (section bar). An @-form surrounding a rewriter limits the scope of the rewriter to within its coordinates. For example, a rewriter applied at (index 1) would transform an object at (index 1), but not at (index 2).

If an object is surrounded by multiple @-forms, the coordinates of the two @-forms are *merged*. A different merge rule can be defined for each pair of coordinates. Merge rules are a key part of the semantics of Art languages, though no formal semantics has yet been published. Without getting too caught up in details, the following reduction rules show how merges are performed for section × section and index × index.

```
(@ ((section ido ...)) (@ ((section idi ...)) ae ...)) → [section]
(@ ((section ido ... idi ...)) ae ...)
(@ ((index no ...)) (@ ((index ni ...)) ae ...)) → [index]
(@ ((index no ... ni ...)) ae ...)
```

The two rules have essentially the same form- **section** appends the section names to create a more nested section, while **index** appends the indices to create a higher dimension index. Other merge rules exist, but they will not be discussed here. These are the only two rules required to understand this document.

Below are a few administrative rewriters we will make use of which handle coordinates and embeddings.

```
<rewriter> ::= ( zoom )
              | ( rewrite-in-music <rewriter>* )
```

The **zoom** rewriter filters out all objects not within its coordinates. A **rewrite-in-music** or **rewrite-in-script** rewriter is a higher-order rewriter that applies its contained rewriters inside each **music** or **script** embedding, respectively, that appears within its coordinates.

### 3.2 Zooming in on a Concert Program

We will use the defined rewriters to zoom in on and play back part of the PorchFest program.

First, we zoom in on the **row-row-row** section of the program.

```
(realize (scribble-realizer)
 porchfest-program
 (@ [(section look-mom row-row-row)] (zoom)))
```

#### Row Row Row Your Boat.

Cat gives an introduction.



Observe the score. This score actually specifies 3 parts. Each %, called a *Segno* (Italian for "sign"), indicates an entrance of a new part. One performer is to begin playing, and whenever a % is reached, a new performer is to come in from the beginning with the same melody. This is known in music as a "simple canon" or a "round". The nursery rhyme "Row Row Row Your Boat" is carefully crafted in order to work in this manner.

Having successfully zoomed in on this snippet, the goal is to play it back digitally. In traditional workflows, this snippet would be a rendered image and therefore surely could not be played back. However, even supposing one pasted this into a score editor or developed an editor for concert programs that embeds a traditional score editor, playing this back would require the score editor's playback engine to have knowledge of the meaning of the % in this context (it can have a handful of meanings in music). It is not desirable to hard-wire this behavior into the playback engine, so we find ourselves effectively two steps removed from the desired outcome.

In our Programmart program, this music snippet is written in embedded Tonart. Now, Tonart's standard facilities for playback do not understand the meaning of the %, either.

However, Art languages come equipped with a rewriting system designed for exactly this scenario. Tonart in particular has a rewriter called `expand-canon` which expands the condensed score expressed via `%` into the full three-voice version. We can apply `expand-canon` to this section by using `zoom` and `rewrite-in-music`. The subforms of `expand-canon` are the names of the new voices that will be introduced by the expansion:

```
(realize (scribble-realizer)
  porchfest-program
  (@ [(section look-mom row-row-row)] (zoom))
  (rewrite-in-music (expand-canon two three)))
```

### Row Row Row Your Boat.

*Cat gives an introduction.*

The transformed score can be played back in Tonart using any stock realizer that can interpret music notes as digital audio. Moreover, the transformed score could be realized into Scribble, printed, and handed out as a performance aid.

### 3.3 Maintaining Multiple Versions of a Concert Program

In many cases it is desirable to have multiple different versions of the same concert materials. Different versions of the same concert might be given on different days. The choir may receive a different version of the program than the congregation in a church service. One version may have the music listed with only title and composer, while another version will have the music printed directly in the program.

For the PorchFest concert, the program was used as a practice aid, which naturally gave rise to a multi-version scenario. This is because a violinist typically reads music in treble clef, a violist typically reads music in alto clef, and a cellist typically reads music in bass clef. Doing this with a traditional workflow would involve maintaining three parallel copies of all materials in each clef, which will have to be kept in sync if any edits are made to the program content.

In Programmart, the desired clef can be composed at the top-level with the score, and all examples will assume that clef. Multiple definitions can reference `porchfest-program`, permitting the following configuration:

```
(define-art porchfest-program-violin
  porchfest-program (clef treble))
(define-art porchfest-program-violola
  porchfest-program (clef alto))
(define-art porchfest-program-cello
  porchfest-program (clef bass))
```

Here is *Hot Cross Buns* rendered in each of the three clefs:

#### Hot Cross Buns.

*Jared gives an introduction.*

#### Hot Cross Buns.

*Jared gives an introduction.*

#### Hot Cross Buns.

*Jared gives an introduction.*

Any changes to `porchfest-program` propagate to all three versions automatically.

### 3.4 Composing Programs

An enjoyable property of Art languages is their composability. In Tonart, scores are composed simply by listing them one after the other. The beats and voices are overlaid naturally and consequent rewrites can touch elements from both scores without any additional structural information. Programmart has similar properties. We will leverage them to improve the look of the PorchFest program without touching the original program source.

#### 3.4.1 New Content, Unordered.

First, we will endeavor to add a background color to each section. A new object, `bg`, will be used to represent a background color. The concert program to compose is rather abstract, consisting purely of background colors:

```
(define-art color-program
  (@ [(section look-mom)]
    (bg "mistyrose"))
  (@ [(section twinkle)]
    (bg "yellow"))
  (@ [(section hot-cross)]
    (bg "tan")))
```

```
(@ [(section row-row-row)]
  (bg "lightblue"))))
```

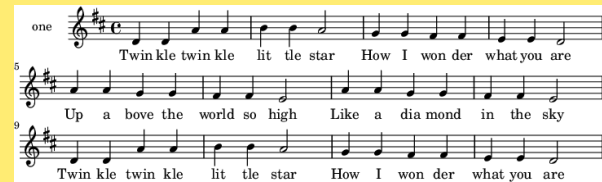
We compose it with the PorchFest program and zoom in on the relevant section:

```
(realize (scribble-realizer)
  porchfest-program
  color-program
  (@ [(section look-mom)] (zoom)))
```

### Look mom, I can play a string instrument

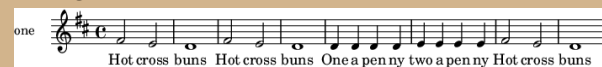
#### Twinkle Twinkle Little Star

*Maria gives an introduction.*



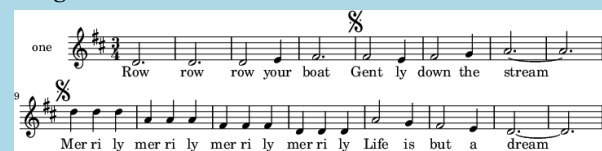
#### Hot Cross Buns

*Jared gives an introduction.*



#### Row Row Row Your Boat

*Cat gives an introduction.*



The two overlap naturally to produce a colored program. Note that this particular case is made simpler by the fact that a background color is not positional content and therefore does not need an index.

### 3.4.2 New Content, Ordered.

Suppose then that we wanted to add a small graphic at the end of each subsection, to illuminate the nursery rhymes. If we use the prior method, no ordering is guaranteed, and the graphic could end up anywhere within the subsection. Each respective graphic therefore needs to be ascribed an index that is greater than the existing greatest index in each respective section.

We start with this standalone concert program, made up only of graphics. This introduces a new object called `image`, which contains a path to an image.

```
(define-art graphics-program
  (@ [(section look-mom twinkle)]
    (image "graphics/stars-row.png"))
  (@ [(section look-mom hot-cross)]
```

```
(image "graphics/buns-grid.png"))
  (@ [(section look-mom row-row-row)]
    (image "graphics/boat.png"))))
```

One might naively like to ascribe each image its own index which is sufficiently large to put it at the end of the section, when composed with the PorchFest program. However, this is not very composable or modular. The goal will be to correctly compose `graphics-program` with `porchfest-program` without changing the source of either.

There are multiple ways to accomplish this. One is to define a new rewriter that performs a merge of the two programs, which is perhaps the most customizable solution. Another interesting solution is detailed below:

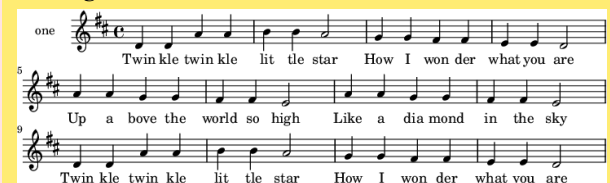
```
(realize (scribble-realizer)
  (ix@ 0 porchfest-program)
  (ix@ 1 graphics-program)
  (@ [(section look-mom)] (zoom)))
```

This solution uses the merge behavior of the `index` coordinate—it composes the programs by adding a dimension to every `index`. In doing so we guarantee that the elements of `graphics-program` come strictly after the elements of `porchfest-program` in each section.

### Look mom, I can play a string instrument

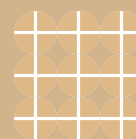
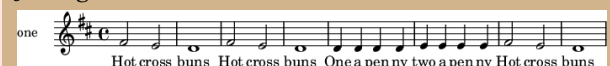
#### Twinkle Twinkle Little Star

*Maria gives an introduction.*



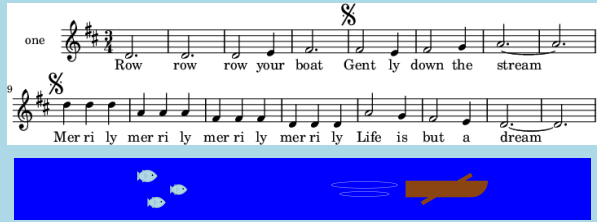
#### Hot Cross Buns

*Jared gives an introduction.*





**Row Row Row Your Boat**  
Cat gives an introduction.



A graphic has been added to each section, giving the program a more wholesome feel.

### 3.4.3 New Section.

Finally, we will add a new nursery rhyme to the concert program. To do this we'll write a separate concert program containing only "Mary had a Little Lamb".

```
(define-art little-lamb-section
  (@ [(section little-lamb)]
    (ix--
      (art-title "Mary Had a Little Lamb")
      (bg "lavender")
      (script (stage-direction "Mariette gives
an introduction."))
      (image "graphics/lamb.png")
      (music mary-had-a-little-lamb))))
```

Sadly we cannot just drop `little-lamb-section` into the look-mom section. The sections that get displayed are controlled by a contextual `layout` object. A section that is not in the layout is not going to be displayed.

```
(define-art porchfest-program
  (layout look-mom dream-sequence celebration)
  (@ [(section look-mom)]
    (layout twinkle hot-cross row-row-row)
    ...)
  (@ [(section dream-sequence)] ...)
  (@ [(section celebration)] ...))
```

To correctly add this section to the program layout, we use the rewriter `append-section`. This rewriter expects an identifier and an `#:after` anchor identifier as subforms. It then looks up the `layout` of the section it is applied to, and appends the identifier to the layout after the anchor.

```
(realize (scribble-realizer)
  (ix@ 0 porchfest-program)
  (ix@ 1 graphics-program)
  color-program
  (@ [(section look-mom)]
    little-lamb-section
    (append-section little-lamb
      #:after hot-cross))
```

(zoom)))

After the rewrite, look-mom's layout is:

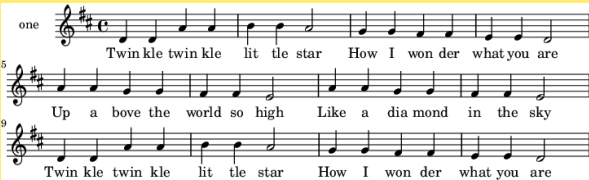
(layout twinkle hot-cross **little-lamb** row-row-row)

The final rendered section looks as follows:

### Look mom, I can play a string instrument

#### Twinkle Twinkle Little Star

Maria gives an introduction.



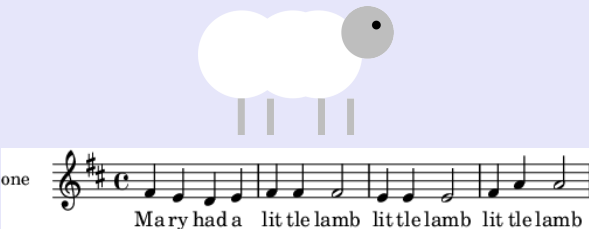
#### Hot Cross Buns

Jared gives an introduction.



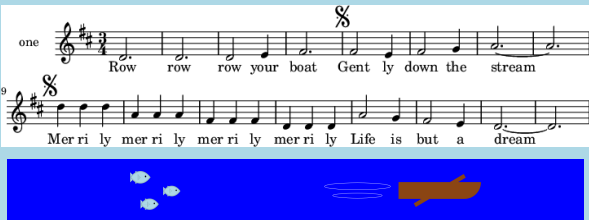
#### Mary Had a Little Lamb

Mariette gives an introduction.



#### Row Row Row Your Boat

Cat gives an introduction.



The new song now fits snugly into the program.

## 4 Conclusion and Discussion

Programmart enables modes of composition and modularity to the area of concert program construction. By embedding Tonart and other Arts, it also assumes the pleasant properties of those languages for itself. It remains open to further embeddings which can coordinate dance, lighting, or other performance disciplines.

To the best of the author's knowledge, no prior system has given such a treatment to concert programs- word processors and publishing software have seemed to be enough, cross-industry, for concert-program-like documents. Whether this indicates a lack of need, or a potential space for development, is unclear at present.

One non-obvious inspiration for Programmart is APL [2], specifically with respect to multi-dimensional indices. Art in fact embeds a small scheme-syntax APL for dealing with multi-dimensional indices, called *ravel* (it takes its name from both the APL operator and the French composer Maurice Ravel).

Here is a basic example using our inserted section:

```
(realize (scribble-realizer ....)
  little-lamb-section
  (run-apl
    (replicate
      (lit 2 3 2 1 1)
      (reverse *ctxt*))))
```

**Mary Had a Little Lamb**

Mariette gives an introduction.  
Mariette gives an introduction.

Those who know APL will recognize the forms of *run-apl* immediately. *reverse* reverses the order of the elements. *replicate* copies each element of the section based on the

numbers in the *lit* array literal. The only novelty is *\*ctxt\**, which is a reference to the concert program.

An important future direction for this work is the building of a graphical editor that can be used to construct concert programs. Prior work on Tonart moved from a textual representation to an embedding into existing score editors [4], which has proven to be a far more efficient workflow. The issue for Programmart is: there are no genuine extant concert program editors to embed into. This hypothetical editor should be easy to use, yet still take advantage of the modularity and composability of Programmart. There is a possibility of making the editor embeddable into DrRacket, using DrRacket's traditional embedded editors, or the more modern Visual Interactive Syntax [5]. This idea was brought up in a prior publication in the context of an embeddable score editor but was rejected, not for any other reason than the sophistication and widespread use of existing score editors, and the maintenance burden of such an implementation. A concert program editor, on the other hand, has fewer needs, suffers from fewer expectations, and can be far less elaborate, while still being acceptable to professionals.

## References

- [1] Nienhuys, Han-Wen and Nieuwenhuizen, Jan. LilyPond, a system for automated music engraving. In *Proc. XIV Colloquium on Musical Informatics*, pp. 167–172, 2003. <https://lilypond.org/>
- [2] Iverson, Kenneth E. Notation as a Tool of Thought. *Communications of the ACM* 23(8), pp. 444–465, 1980. <https://doi.org/10.1145/358896.358899>
- [3] Gentner, Jared. Demo: Composable Compositions with Tonart. In *Proc. 12th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design*, pp. 42–44, 2024. <https://doi.org/10.1145/3677996.3678294>
- [4] Gentner, Jared. Demo: How to Score in the Art Racket. In *Proc. 13th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design*, 2025. <https://doi.org/10.1145/3759162.3759648>
- [5] Andersen, Leif, Ballantyne, Michael, and Felleisen, Matthias. Adding interactive visual syntax to textual code. In *Proc. ACM on Programming Languages*, pp. 1–28, 2020. <https://doi.org/10.1145/3428290>
- [6] Flatt, Matthew, Barzilay, Eli, and Findler, Robert Bruce. Scribble: Closing the Book on Ad Hoc Documentation Tools. In *Proc. 14th ACM SIGPLAN International Conference on Functional Programming*, pp. 109–120, 2009. <https://doi.org/10.1145/1596550.1596569>